

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de



Recap / Vorstellungen

- Aufgabe Mehrfachvererbung noch offen
- Q&A Dokumente, um wieder in die Themen reinzukommen

Kapitel 2: Objektorientierung

21. Grundlagen

Exkurs: Git

C++ Basics: Header Files

22. Vererbung

C++ Basics: Pointer und Referenzen

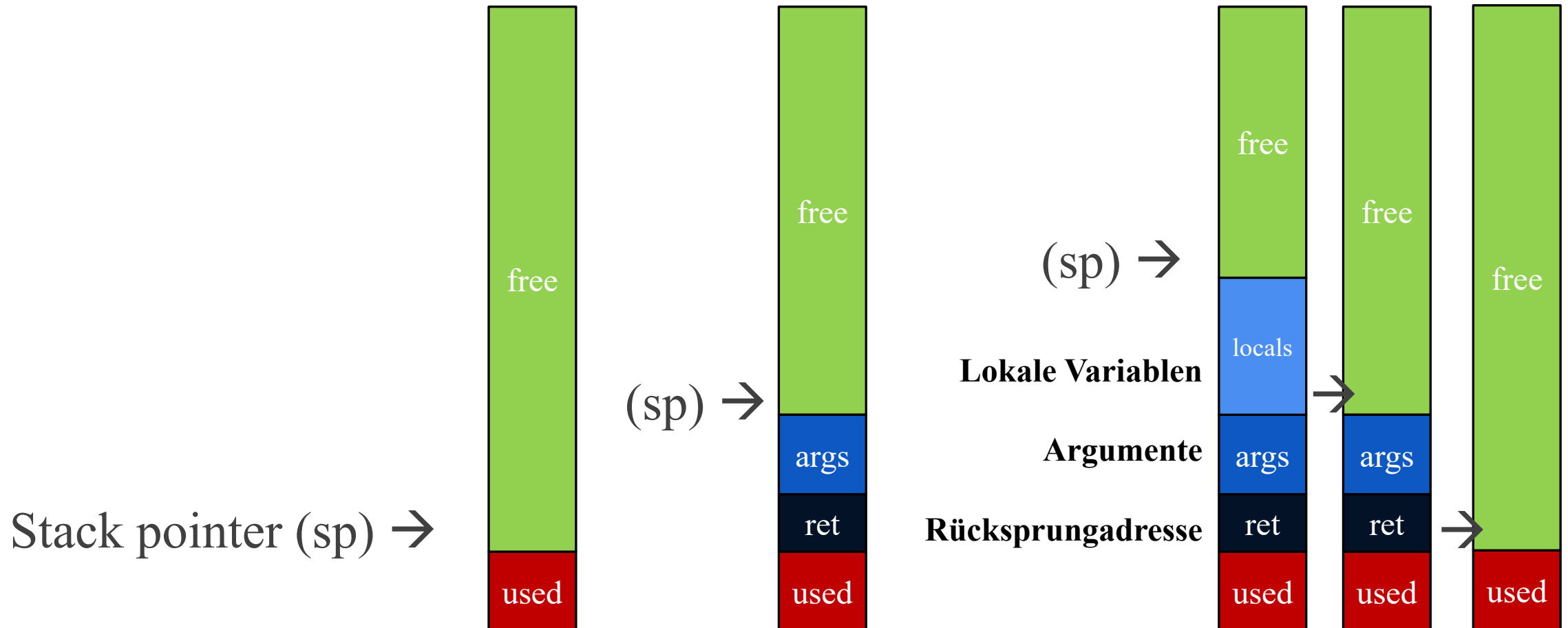
23. Polymorphismus



C++ Basics: Speicherverwaltung

Speicherverwaltung – Stack bei Funktionsaufruf

Jeder Funktionsaufruf legt einen **Stack-Frame** oben drauf



New/delete (formally known as malloc/free – more powerful)

- Keywords `new` und `delete` werden genutzt um Speicher zu allokieren und wieder frei zu geben.
- Auch `new` und `delete` sollten soweit möglich vermieden werden (mehr dazu später)
- Regel: Für jedes `new` ein `delete`

```
double* ptr = new double;           // reserve memory for pointer (on heap)
delete ptr;                          // free reserved memory
ptr = nullptr;                      // set ptr to NULL

ptr = new double(7.0);               // reserve memory for pointer and set value to 7.0
delete ptr;
```

Stack vs. Heap

Stack

- **Verwaltung** - abhängig von Programmiersprache, OS, System-Architektur
- Nicht direkt vom Entwickler beeinflussbar (aber z.B. in Assembler)
- **Freigabe** - von Speicher erfolgt automatisch
- **Größe** - wird bei Programmstart festgelegt
- **Zugriff** - Ohne Pointer nutzbar

VS

Heap

- Muss vom Entwickler manuell verwaltet werden
- Viele Programmiersprachen bringen Garbage-Kollektoren mit, die obsoleten Speicher finden und frei geben
- Manuell vergrößerbar
- Kann nur über Pointer angesprochen werden

```
Class myClass1;           // Stack  
Class* myClass2 = new Class(); // Heap
```

Stack – Fragen

- Was passiert mit dem Stack bei einer Endlosrekursion?
- Was passiert bei einem Infinite-Loop?
- Wenn eine Schleife 10.000 mal durchläuft und darin eine Variable benötigt wird.
Wie viel weniger Speicher wird initialisiert, wenn diese davor 1 mal deklariert wird?

Stack vs. Heap vs. Smart Pointer

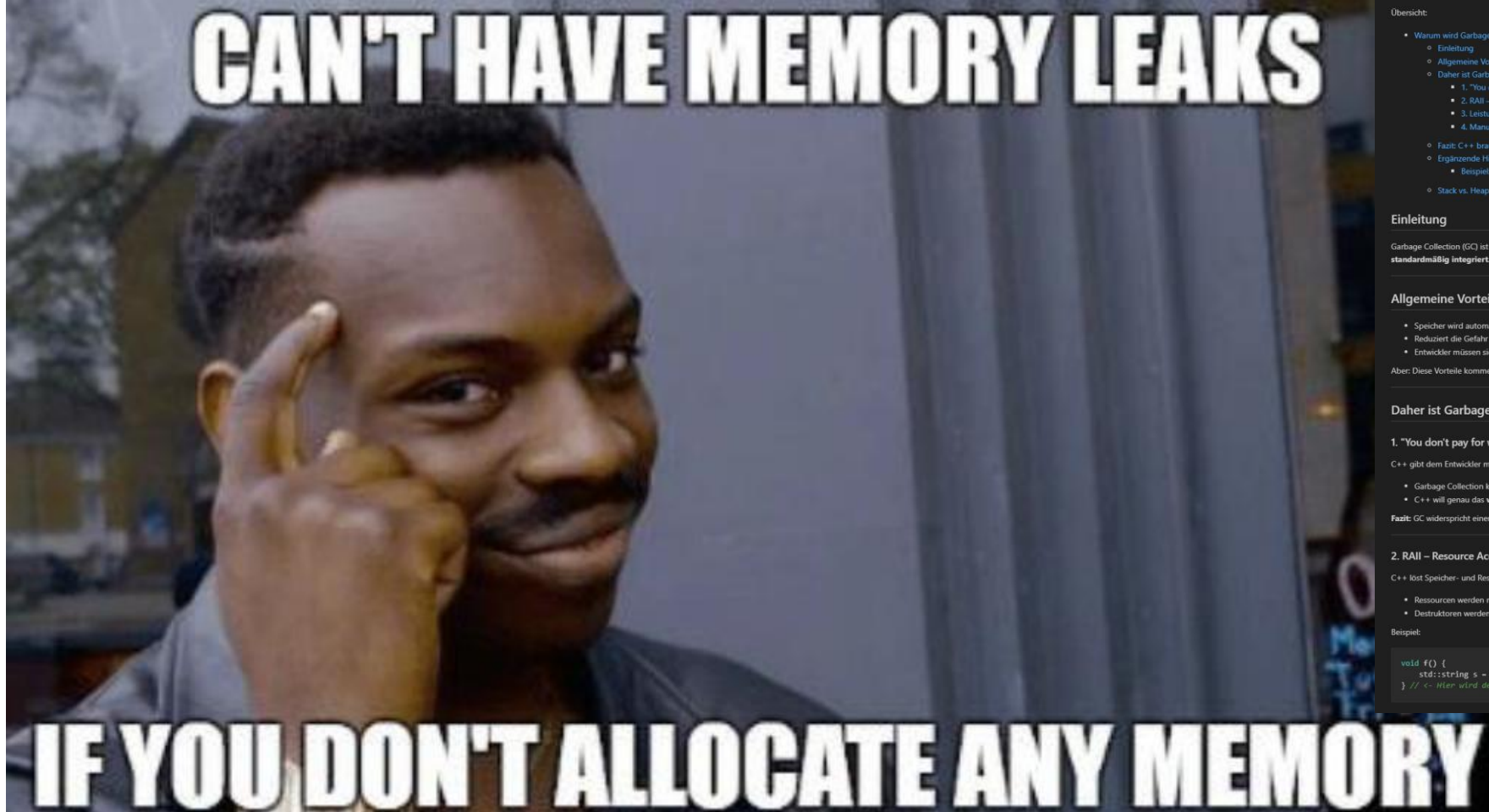
	Stack	Heap (new / delete)	Smart Pointer
Beispiel	<code>double x = 7.0;</code>	<code>new double(7.0)</code>	<code>make_unique<double>(7.0)</code>
Speicherort	Stack	Heap	Heap
Wer räumt auf	der Compiler	du selbst (delete)	der Destruktor
Wann	am Scope-Ende	wenn du delete schreibst	am Scope-Ende
Fehlerrisiko	keins	Leak / dangling pointer	keins
Empfehlung	Standard	nur wenn nötig	wenn Heap nötig

Gleicher Wert, gleicher Endzustand – nur die Verantwortung fürs Aufräumen wandert. Deshalb: Stack bevorzugen, Smart Pointer statt rohem new/delete.

Fazit – Pointer und Speicherverwaltung

- **KISS!**
- Pointer nur benutzen wenn zwingend notwendig (in Legacy code oft nicht vermeidbar)
- Vermeiden Pointer gegenseitig zuzuweisen
- Call-by-Reference immer nutzen falls Argumente nicht trivial
- Embedded Software: Oft gibt es überhaupt keinen Heap!
- Wenn doch mit Pointern gearbeitet werden soll, dann gibt es smart Pointer
`std::unique_ptr<Class>` `std::shared_ptr(new Class())`
Diese kümmern sich um Speicherverwaltung (hier nicht im Detail besprochen).

Fazit – Pointer und Speicherverwaltung



Warum wird Garbage Collection in C++ kaum verwendet?

Übersicht:

- Warum wird Garbage Collection in C++ kaum verwendet?
 - Einleitung
 - Allgemeine Vorteile von Garbage Collection
 - Daher ist Garbage Collection in C++ ungewöhnlich
 - 1. "You don't pay for what you don't use" – Das C++ Prinzip
 - 2. RAII – Resource Acquisition Is Initialization
 - 3. Leistung und Echtzeitfähigkeit
 - 4. Manuelle Kontrolle ist ein Feature, kein Bug
 - Fazit: C++ braucht keine klassische Garbage Collection
 - Ergänzende Hinweise / Alternativen zur GC in C++
 - Beispiel: Speicherleck und Smart Pointer
 - Stack vs. Heap Speicher

Einleitung

Garbage Collection (GC) ist in vielen modernen Programmiersprachen wie Java oder C# ein selbstverständlicher Bestandteil. In C++ hingegen ist sie bewusst **nicht standardmäßig integriert**. Warum?

Allgemeine Vorteile von Garbage Collection

- Speicher wird automatisch freigegeben
- Reduziert die Gefahr von Speicherlecks
- Entwickler müssen sich nicht um Speicherfreigabe kümmern

Aber: Diese Vorteile kommen **nicht ohne Kosten**.

Daher ist Garbage Collection in C++ ungewöhnlich

1. "You don't pay for what you don't use" – Das C++ Prinzip

C++ gibt dem Entwickler maximale Kontrolle und vermeidet unnötige Kosten:

- Garbage Collection kostet **Rechenzeit und Speicher**, selbst wenn nichts bereinigt werden muss
- C++ will genau das **vermeiden**, wenn die Funktion nicht benötigt wird

Fazit: GC widerspricht einem Grundprinzip von C++

2. RAII – Resource Acquisition Is Initialization

C++ löst Speicher- und Ressourcenmanagement durch RAII:

- Ressourcen werden mit Objektlebensdauer verwaltet
- Destruktoren werden **deterministisch** (zu einem klar definierten Zeitpunkt) aufgerufen

Beispiel:

```
void f() {  
    std::string s = "Hello"; // Speicher wird automatisch freigegeben sobald wir den Scope verlassen  
} // <- Hier wird der Speicher freigegeben
```

enum class und switch cases

Aufgaben: Objektorientierung – .hpp Dateien, enum class, switch cases

Erstellen Sie ihre Klassen in .hpp und .cpp Files und rufen sie die Klassen aus einem separaten main.cpp file auf.

1) Erstellen Sie eine Klasse Person.

- a) Die einen Namen und eine Nationalität hat (aus: `de`, `en`, `it`, `es` → `enum class Nationality`)
- b) Die eine `string getName()` Methode zum abrufen des privaten Namens hat.
- c) Die eine Member Funktion `void greet(Person greetedPerson)` hat, die eine Grußformel in der Muttersprache in der Konsole ausgibt, z.B. „Buongiorno Jose“ für `it` und `greetedPerson = Jose` (→ `switch case`).

Kapitel 2: Objektorientierung

21. Grundlagen

22. Vererbung

23. Polymorphismus

Exkurs: Git



C++ Basics: ~~Header Files~~

C++ Basics: Pointer und Referenzen

C++ Basics: Speicherverwaltung

Die `std::vector` Klasse, neue for loops und Referenzen

Aufgaben: Objektorientierung – .hpp Dateien, enum class, switches und for loops (vehicle_example_V2)

Erstellen Sie ihre Klassen in .hpp und .cpp Files und rufen sie die Klassen aus einem separaten main.cpp file auf.

- 1) Erstellen Sie eine Klasse Person.
 - a) Die einen Namen und eine Nationalität hat (aus: de, en, it, es → enum class Nationality)
 - b) Die eine string getName() Funktion zum abrufen des privaten Namens hat.
 - c) Die eine Member Funktion void greet(Person greetedPerson) hat, die eine Grußformel in der Muttersprache in der Konsole ausgibt, z.B. „Buongiorno Jose“ für it und greetedPerson = Jose (→ switch case).
- 2) Erstellen Sie eine neue Vehicle Klasse. Diese soll ...
 - a) Über eine für jede Instanz fixe Anzahl an Sitzen verfügen.
 - b) Eine Liste an Personen haben die gerade im Fahrzeug sitzen.
 - c) Eine Funktion enter(Person person) und eine Funktion exit(int seatNumber) haben.
- 3) Erweitern Sie die enter-Funktion, so dass alle im Fahrzeug anwesenden von der neu einsteigenden Person begrüßt werden und danach alle im Fahrzeug sitzenden Personen die neue Person begrüßen.

Aufgaben: Objektorientierung – .hpp Dateien, enum class, switches und for loops (vehicle_example_V2)

4) Zusatzaufgabe (wird nicht besprochen)

- a) Erweitern Sie die Klasse Vehicle um eine Farbe aus: blau, grün, gelb, rot → enum class
- b) Geben Sie die Begrüßung in der jeweiligen Farbe auf der Konsole aus
- c) Erstellen Sie eine Klasse TollStation mit einer funktion control und einer Person cashier, welche eine Liste von Fahrzeugen kontrolliert. Dabei begrüßen alle Insassen aus jedem Fahrzeug in der jeweiligen Farbe den cashier (füge dazu eine weitere funktion greetAll zur Klasse Vehicle hinzu). Außerdem wird der Preis der Mautstelle ausgegeben